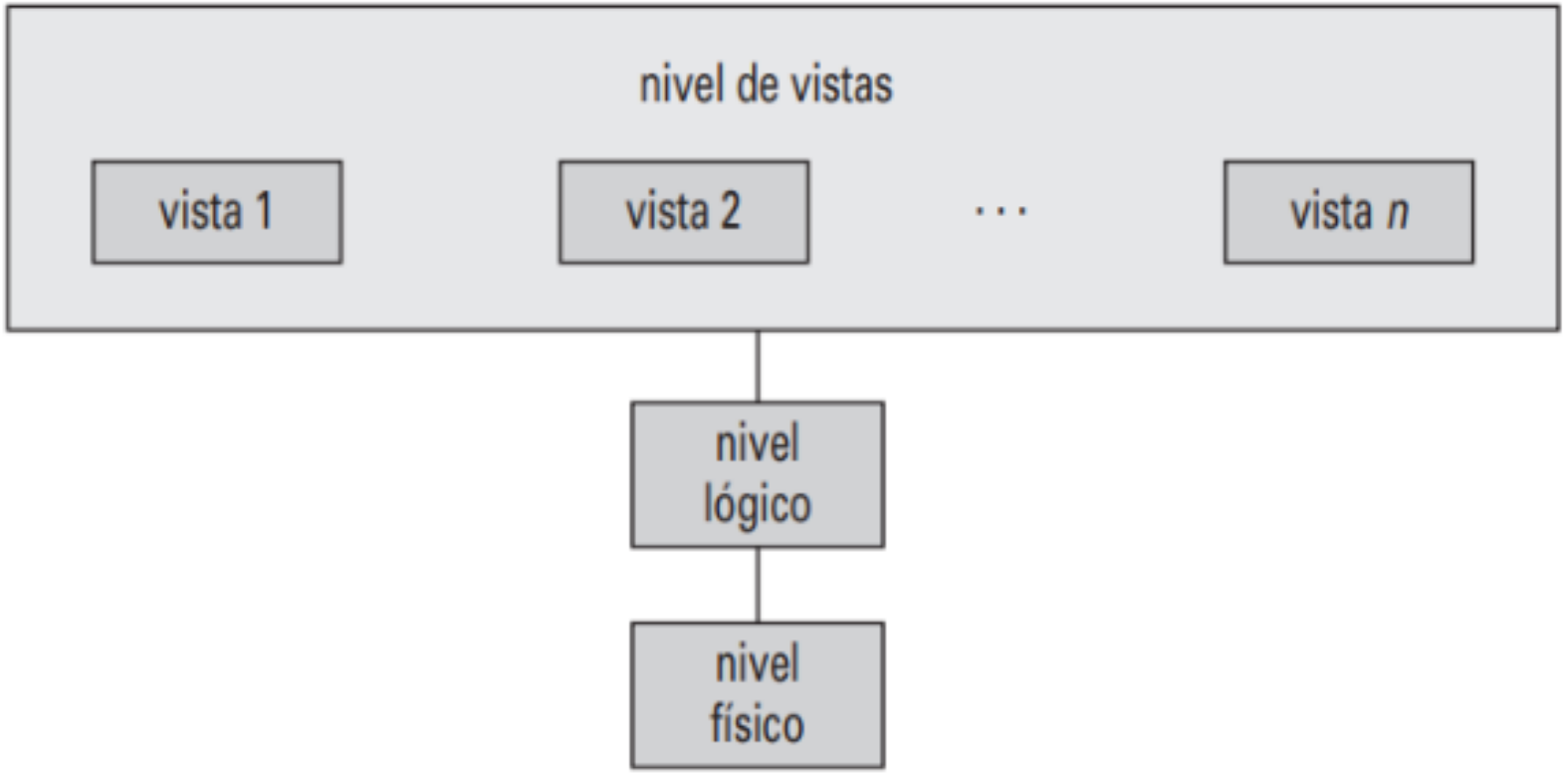
Arquitectura ANSI/SPARC



Vistas

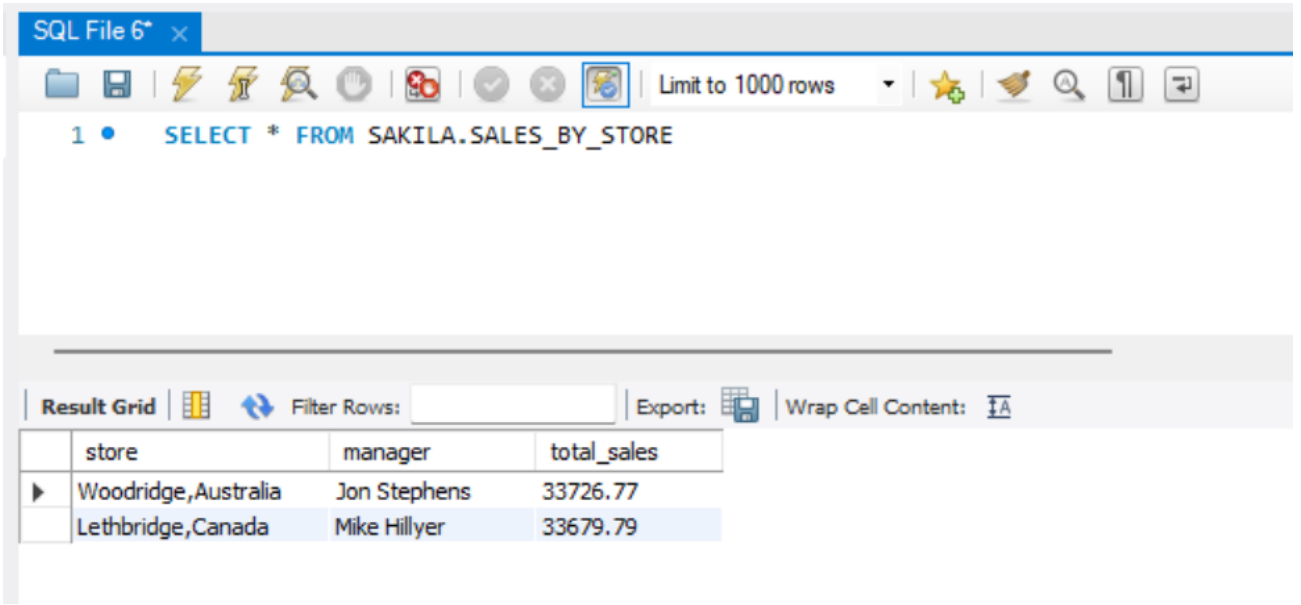
- **Definición**
- **Ventajas de las vistas**
- **Creación de vistas**
- **Restricciones en la creación de vistas**
- **Modificación de vistas**
- **Eliminación de vistas**
- **Modificación de datos a través de vistas**

¿Qué es una vista?

No es deseable que todos los usuarios puedan ver la totalidad del modelo lógico. Las consideraciones sobre la seguridad pueden exigir

una **vista** es una **relación virtual**

una vista **no almacena los datos físicamente en disco**. En su lugar, almacena únicamente una consulta SQL. Cada vez que la vista se consulta, el motor ejecuta la consulta subyacente para generar los datos al vuelo.



Vistas - Ventajas

- **Seguridad y privacidad (Ocultamiento de datos):** Permite mostrar solo las columnas o filas a las que un usuario tiene derecho.
- **Simplificación de consultas:** Encapsula lógica compleja (JOINs múltiples, agregaciones, subconsultas) bajo una interfaz limpia de tabla única.
- **Independencia Lógica de los Datos:** Si la estructura interna de las tablas base cambia (se renombran o dividen columnas), la vista se puede rediseñar para que la aplicación del usuario siga funcionando sin modificaciones.
- **Personalización para distintos perfiles:** Un mismo modelo relacional puede ofrecer diferentes "caras" o vistas a la medida de la necesidad de cada departamento (Ventas, RRHH, Finanzas).

Vistas: Creación

```
CREATE VIEW nombre_vista AS (
    SELECT columna1,
    columna2,
    ...
    FROM tabla
    WHERE condicion );
```

```
CREATE VIEW sakila.sales_by_store AS (
SELECT concat(c.city,cy.country) AS store,
concat(m.first_name,m.last_name) AS manager,
sum(p.amount) AS total_sales
FROM sakila.payment p
INNER JOIN sakila.rental r ON (p.rental_id = r.rental_id)
INNER JOIN sakila.inventory i ON (r.inventory_id = i.inventory_id)
INNER JOIN sakila.store s ON (i.store_id = s.store_id)
INNER JOIN sakila.address a ON (s.address_id = a.address_id)
INNER JOIN sakila.city c ON (a.city_id = c.city_id)
INNER JOIN sakila.country cy ON (c.country_id = cy.country_id)
INNER JOIN sakila.staff m ON (s.manager_staff_id = m.staff_id)
GROUP BY s.store_id
ORDER BY cy.country,c.city)
```

EJERCICIO

El gerente del videoclub Sakila necesita que el equipo de atención al cliente tenga acceso rápido al catálogo de películas de **Acción** sin exponer campos confidenciales o internos de la base de datos (como costos de reemplazo o tarifas de alquiler).

Además, el equipo no sabe escribir JOINs complejos, por lo que necesitan consultar esta información como si fuera una simple tabla.

```
CREATE VIEW nombre_vista AS (
    SELECT columna1,
    columna2,
    ...
    FROM tabla
    WHERE condicion );
```

Creación Vistas - Restricciones

Al definir una vista, hay reglas estrictas que se deben respetar:

- **Nombres de columnas duplicados o calculados:** Si la consulta SELECT incluye expresiones (ej. precio * 1.21) o dos tablas con el mismo nombre de columna, es **obligatorio asignar alias** (AS nombre_columna).

- **Dependencia de objetos:** Una vista no puede crearse si hace referencia a tablas o vistas inexistentes. Si borras la tabla base original, la vista queda en estado inválido.

- **Restricciones de ordenamiento:** En el estándar SQL puro no se permite la cláusula ORDER BY dentro de la definición de la vista (aunque algunos motores modernos como MySQL lo permiten bajo ciertas circunstancias).

Vistas: Modificación - Eliminacion

```
CREATE OR REPLACE VIEW nombre_vista AS
SELECT nueva_columna1, nueva_columna2
FROM tabla;
```

Modificación de la definición de la vista
Si necesitas cambiar la consulta de la vista sin perder permisos asignados a ella

```
DROP VIEW nombre_vista;
```

Eliminación de la vista
Para borrar la estructura

CONSIGNA:

EJERCICIO

- El gerente avisa que la vista se quedó corta. Ahora el equipo de atención al cliente necesita ver también la **duración en minutos** de la película (length).

- ¿Al ejecutar DROP VIEW v_catalogo_accion; se borraron las películas de acción de la tabla film?